

Caso de Estudio

Reconstrucción de Señales de Voz en Entornos Ruidosos

mediante el Filtro de Kalman

Juan Bautista Espino

Modelado y Simulación — 2026, 1^{er} cuatrimestre

24 de abril de 2026

Índice

1. Planteamiento del Problema	2
2. Objetivo e Hipótesis	2
2.1. Objetivo	2
2.2. Hipótesis	2
3. Marco Teórico	2
3.1. Modelo de señal observada	2
3.2. Filtro de Kalman escalar	3
3.3. Métodos clásicos de referencia	3
3.4. Métricas de evaluación	4
4. Metodología	4
4.1. Generación de la señal de voz sintética	4
4.2. Adición de ruido blanco gaussiano	5
4.3. Pipeline de comparación	5
4.4. Implementación en Python	5
5. Resultados	6
5.1. Configuración del experimento	6
5.2. Métricas obtenidas	6
5.3. Análisis de resultados	6
6. Discusión	7
6.1. ¿Por qué el Filtro de Kalman supera a la interpolación?	7
6.2. Rol de los parámetros Q y R	7
6.3. Limitaciones del modelo de paseo aleatorio	7
7. Conclusiones	8

1. Planteamiento del Problema

En aplicaciones reales de comunicación y procesamiento del habla —clases virtuales, llamadas telefónicas, sistemas de reconocimiento automático de voz (ASR)—, la señal captada por el micrófono no es la voz pura del locutor sino una versión degradada por:

- **Ruido ambiental:** tráfico, viento, conversaciones de fondo.
- **Ruido electrónico:** cuantificación del ADC, interferencias del circuito.
- **Pérdidas de paquetes:** en comunicación VoIP o streaming.

Estas perturbaciones reducen la inteligibilidad, dificultan el análisis automático y degradan la experiencia perceptual del oyente.

Pregunta central

¿Cómo reconstruir una señal de voz más cercana a la original a partir de mediciones ruidosas, usando herramientas de estimación estadística?

2. Objetivo e Hipótesis

2.1. Objetivo

Evaluar cuantitativamente si el filtro de Kalman logra:

1. Reducir el ruido aditivo en señales de voz sintéticas.
2. Aproximar mejor la señal “real” que métodos de interpolación clásicos.
3. Ofrecer una ventaja medible en las métricas MSE y SNR.

2.2. Hipótesis

El filtro de Kalman proporciona una reconstrucción más fiel de la señal de voz que la interpolación polinómica y el spline cúbico, gracias a su capacidad de modelar explícitamente la incertidumbre del proceso y de la medición, e incorporar la dinámica temporal de la señal.

3. Marco Teórico

3.1. Modelo de señal observada

Se modela la situación como un sistema de estado discreto:

$$x_k = x_{k-1} + w_k, \quad w_k \sim \mathcal{N}(0, Q) \quad (1)$$

$$z_k = x_k + v_k, \quad v_k \sim \mathcal{N}(0, R) \quad (2)$$

donde x_k es la amplitud real (desconocida) de la voz en el instante k , z_k es la muestra ruidosa observada, w_k es el ruido de proceso (variabilidad intrínseca de la señal) y v_k es el ruido de medición (ruido ambiental/electrónico). Las varianzas Q y R cuantifican la incertidumbre en cada fuente.

3.2. Filtro de Kalman escalar

El filtro de Kalman resuelve el problema de **estimación MMSE** (Minimum Mean Square Error) para el modelo lineal gaussiano de las ecuaciones (1)–(2). Opera en dos pasos recursivos:

Paso 1 — Predicción

Se proyecta el estado y la covarianza del error al instante siguiente sin utilizar aún la medición z_k :

$$\hat{x}_{k|k-1} = \hat{x}_{k-1|k-1} \quad (3)$$

$$P_{k|k-1} = P_{k-1|k-1} + Q \quad (4)$$

Paso 2 — Corrección

Se incorpora la medición z_k ponderando por la **ganancia de Kalman** K_k :

$$K_k = \frac{P_{k|k-1}}{P_{k|k-1} + R} \quad (5)$$

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k(z_k - \hat{x}_{k|k-1}) \quad (6)$$

$$P_{k|k} = (1 - K_k) P_{k|k-1} \quad (7)$$

Interpretación de la ganancia. Si $R \ll P_{k|k-1}$ (alta confianza en la medición), entonces $K_k \rightarrow 1$ y el filtro sigue de cerca a z_k . Si $Q \ll R$ (proceso muy suave y medición muy ruidosa), entonces $K_k \rightarrow 0$ y el filtro ignora la medición y confía en su predicción.

3.3. Métodos clásicos de referencia

Interpolación polinómica

Dado el conjunto de puntos $\{(t_k, z_k)\}_{k=0}^{N-1}$, se busca el polinomio de grado n que minimiza el error cuadrático total (ajuste por mínimos cuadrados):

$$P(t) = \sum_{j=0}^n c_j t^j, \quad \mathbf{c} = \arg \min_{\mathbf{c}} \sum_{k=0}^{N-1} (z_k - P(t_k))^2 \quad (8)$$

Para grados altos y datos equiespaciados aparecen las **oscilaciones de Runge** en los extremos del intervalo, que amplifican el error en lugar de reducirlo.

Spline cúbico natural

Se divide el dominio en subintervalos mediante nodos $\{t_{i_0}, t_{i_1}, \dots\}$ obtenidos por submuestreo (cada s muestras). En cada subintervalo se ajusta un polinomio cúbico, imponiendo continuidad de primera y segunda derivada:

$$S_j(t) = a_j + b_j(t - t_j) + c_j(t - t_j)^2 + d_j(t - t_j)^3, \quad t \in [t_j, t_{j+1}] \quad (9)$$

con condiciones de frontera naturales $S''(t_0) = S''(t_{N-1}) = 0$.

3.4. Métricas de evaluación

$$\text{MSE} = \frac{1}{N} \sum_{k=1}^N (x_k - \hat{x}_k)^2 \quad (10)$$

$$\text{SNR} = 10 \log_{10} \left(\frac{\sum_k x_k^2}{\sum_k (x_k - \hat{x}_k)^2} \right) \quad [\text{dB}] \quad (11)$$

Un MSE menor y un SNR mayor indican mejor reconstrucción.

4. Metodología

4.1. Generación de la señal de voz sintética

Se genera una señal tipo voz mediante una suma de $H = 7$ armónicos con amplitudes decrecientes (modelo de fuente glótica simplificado) y una envolvente de amplitud lenta:

$$x(t) = A(t) \cdot \sum_{k=1}^H a_k \sin(2\pi k f_0 t + \varphi_k), \quad A(t) = \frac{1}{2} \left(1 + \sin(2\pi \cdot 1,5 \cdot t + \frac{\pi}{4}) \right) \quad (12)$$

con $f_0 = 150$ Hz (frecuencia fundamental), $a_k = \{1, 0,6, 0,35, 0,2, 0,12, 0,07, 0,04\}$ y fases φ_k aleatorias uniformes en $[0, 2\pi)$. La señal se normaliza al rango $[-1, 1]$ y se muestrea a $f_s = 8000$ Hz.

4.2. Adición de ruido blanco gaussiano

Para simular un entorno con relación señal/ruido de entrada SNR_{in} dB, se añade ruido blanco $v \sim \mathcal{N}(0, \sigma_v^2)$ donde:

$$\sigma_v^2 = \frac{P_x}{10^{\text{SNR}_{\text{in}}/10}}, \quad P_x = \frac{1}{N} \sum_k x_k^2 \quad (13)$$

4.3. Pipeline de comparación

1. Generar x (señal limpia) y $z = x + v$ (señal ruidosa).
2. Aplicar **interpolación polinómica** (grado $n = 8$) sobre z .
3. Aplicar **spline cúbico** (nodos cada $s = 5$ muestras) sobre z .
4. Aplicar **filtro de Kalman** ($Q = 10^{-2}$, $R = 10^{-2}$) sobre z .
5. Calcular MSE y SNR de salida de cada método comparando con x .

4.4. Implementación en Python

El módulo `kalman_voz.py` implementa todos los métodos. A continuación se muestra el núcleo del filtro de Kalman:

Listing 1: Filtro de Kalman escalar — `kalman_voz.py`

```

1 def filtro_kalman(z, Q=1e-2, R=1e-2, x0=0.0, P0=1.0):
2     n = len(z)
3     x_hat = np.zeros(n)
4     x_pred = x0
5     P = P0
6
7     for k in range(n):
8         # -- Prediccion --
9         P_pred = P + Q
10
11        # -- Correccion --
12        K = P_pred / (P_pred + R)          # Ganancia de Kalman
13        x_hat[k] = x_pred + K * (z[k] - x_pred)
14        P = (1 - K) * P_pred
15
16        x_pred = x_hat[k]
17
18    return x_hat

```

La generación de la señal sintética:

Listing 2: Generación de voz sintética — `kalman_voz.py`

```

1 def generar_senal_voz(n_muestras=400, fs=8000.0, freq_fundamental
  =150.0, seed=42):
2     rng = np.random.default_rng(seed)
3     t = np.linspace(0, n_muestras / fs, n_muestras, endpoint=False)
4
5     x = np.zeros(n_muestras)
6     for k, amp in enumerate([1.0, 0.6, 0.35, 0.2, 0.12, 0.07,
7                             0.04], start=1):
8         phase = rng.uniform(0, 2 * np.pi)
9         x += amp * np.sin(2 * np.pi * k * freq_fundamental * t +
10                          phase)
11
12     env = 0.5 * (1 + np.sin(2 * np.pi * 1.5 * t + np.pi / 4))
13     x *= env
14     x /= np.max(np.abs(x) + 1e-12)
15     return t, x

```

5. Resultados

5.1. Configuración del experimento

Parámetro	Valor
Muestras (N)	400
Frecuencia de muestreo (f_s)	8 000 Hz
Frecuencia fundamental (f_0)	150 Hz
SNR de entrada	10 dB
Grado del polinomio	8
Factor de submuestreo (spline)	5
Q (Kalman)	10^{-2}
R (Kalman)	10^{-2}

5.2. Métricas obtenidas

Método	MSE	SNR de salida (dB)
Señal ruidosa (sin filtrar)	0.016644	10.43
Interpolación polinómica	0.169189	0.36
Spline cúbico	0.017644	10.18
Filtro de Kalman	0.013317	11.79

5.3. Análisis de resultados

Interpolación polinómica. El polinomio de grado 8 presenta el peor desempeño: MSE = 0,169, SNR = 0,36 dB. Al ajustar globalmente sobre los datos ruidosos, el método in-

introduce las oscilaciones de Runge en los extremos del intervalo, *amplificando* el error en lugar de reducirlo. Este comportamiento confirma que la interpolación global es inadecuada para señales estocásticas con ruido.

Spline cúbico. Al utilizar nodos submuestreados (1 de cada 5 muestras), el spline evita interpolar exactamente el ruido, obteniendo $\text{MSE} = 0,018$ y $\text{SNR} = 10,18$ dB, levemente inferior a la señal ruidosa sin procesar. Aunque el suavizado local es conceptualmente correcto, el método no tiene mecanismo para estimar cuánto del residuo es ruido real versus variación legítima de la señal.

Filtro de Kalman. Con $Q = R = 10^{-2}$, el filtro obtiene $\text{MSE} = 0,013$ y $\text{SNR} = 11,79$ dB, superando a todos los métodos. La mejora de +1,36 dB sobre la señal ruidosa y de +1,61 dB sobre el spline es consecuencia directa de la estimación probabilística: en cada paso, el filtro pondera la predicción del modelo con la observación actual según la relación $K_k = P_{k|k-1} / (P_{k|k-1} + R)$, asignando mayor peso a la fuente más confiable.

6. Discusión

6.1. ¿Por qué el Filtro de Kalman supera a la interpolación?

Los métodos de interpolación son esencialmente **deterministas**: buscan una curva que pase “cerca” de los datos observados, sin distinguir si la variabilidad es parte de la señal o ruido. El filtro de Kalman, en cambio, opera en un marco **probabilístico**: modela explícitamente dos fuentes de incertidumbre (Q y R) y computa en cada instante la estimación que minimiza el error cuadrático esperado.

6.2. Rol de los parámetros Q y R

La razón Q/R controla el comportamiento del filtro:

- $Q \ll R$: el filtro confía más en su predicción que en la medición. Produce una salida muy suavizada pero potencialmente retrasada o sesgada.
- $Q \gg R$: el filtro sigue de cerca cada muestra nueva. En el límite ($R \rightarrow 0$), la salida coincide con la señal ruidosa.
- $Q \approx R$: equilibrio óptimo para señales moderadamente no estacionarias con SNR de entrada ~ 10 dB.

6.3. Limitaciones del modelo de paseo aleatorio

El modelo de estado (1) asume que la variación de x_k entre muestras consecutivas es ruido blanco. Para señales de voz reales, esto es una aproximación: la voz tiene correlación de largo plazo (prosodia, vocales sostenidas) que un modelo autoregresivo (AR) o un Kalman

de orden superior capturaría mejor. Sin embargo, para el propósito de este caso de estudio, el modelo escalar demuestra con claridad las ventajas del enfoque probabilístico.

7. Conclusiones

1. El filtro de Kalman escalar, con parámetros $Q = R = 10^{-2}$, logró **SNR** = 11,79 dB, superando la señal ruidosa (10,43 dB), el spline cúbico (10,18 dB) y la interpolación polinómica (0,36 dB).
2. La interpolación polinómica global es **contraindicada** para señales estocásticas con ruido: las oscilaciones de Runge empeoran la reconstrucción respecto a no aplicar ningún filtrado.
3. El spline cúbico ofrece una mejora marginal frente a la señal sin procesar, pero no puede explotar la estructura probabilística del problema.
4. La hipótesis planteada queda **confirmada**: el filtro de Kalman supera a los métodos clásicos gracias a su modelado explícito del ruido de proceso y de medición, y a su actualización recursiva que incorpora la dinámica temporal.
5. Las líneas de trabajo futuro incluyen: modelos AR de orden superior, filtro de Kalman extendido para señales no lineales, e integración con pipelines de reconocimiento automático de voz (ASR).

“Mientras los métodos de interpolación intentan describir los datos observados, el filtro de Kalman busca inferir la realidad que los genera.”

Referencias

1. Kalman, R. E. (1960). *A new approach to linear filtering and prediction problems*. Journal of Basic Engineering, 82(1), 35–45.
2. Haykin, S. (2001). *Kalman Filtering and Neural Networks*. Wiley-Interscience.
3. Proakis, J. G., & Manolakis, D. G. (2006). *Digital Signal Processing: Principles, Algorithms, and Applications* (4th ed.). Prentice Hall.
4. Virtanen, P. et al. (2020). *SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python*. Nature Methods, 17, 261–272.